



MANIPAL

ACADEMY of HIGHER EDUCATION

(Institution of Eminence Deemed to be University)

MANIPAL SCHOOL OF INFORMATION SCIENCES

(A Constituent unit of MAHE, Manipal)

LOS THEORY – Assignment 4

Reg. Number	Name	Branch
261100690024	K P NIDHI	CyberSecurity



MANIPAL SCHOOL OF INFORMATION SCIENCES
MANIPAL
(A constituent unit of MAHE, Manipal)

Linux Filesystem Links and Data Processing

1. Theoretical Background

To understand the behavior of the Linux filesystem, two primary linking mechanisms must be distinguished:

Hard Links

A hard link is a directory entry that associates a name with a specific Inode on a disk. Since all hard links to a file point to the same Inode, they are indistinguishable from the original file. Deleting the original name does not delete the data as long as at least one hard link remains.

Symbolic (Soft) Links

A symbolic link is a special type of file that contains a text string representing a path to another file or directory. It acts like a shortcut. If the target file is moved or renamed, the link becomes 'broken' or 'dangling' because the stored path no longer points to a valid destination.

The Inode System

The Inode (Index Node) is a data structure on a Linux filesystem that stores all the metadata about a file except its name and the actual data. This includes permissions, owner, size, and location on the disk.

2. Part A & B: Link Creation and Metadata Analysis

In this phase, we established the workspace 'link_lab' and initialized a file with confidential security content.

```
(kali@kali)-[/home/kali/link_lab]
PS> printf "Linux File System Security Lab\nConfidential Security Report\n"
> security.txt

(kali@kali)-[/home/kali/link_lab]
PS> cat security.txt
Linux File System Security Lab
Confidential Security Report

(kali@kali)-[/home/kali/link_lab]
PS> ln security.txt security_hard.txt

(kali@kali)-[/home/kali/link_lab]
PS> ln -s security.txt security_soft.txt

(kali@kali)-[/home/kali/link_lab]
PS> ls -li
total 8
786937 -rw-rw-r-- 2 kali kali 60 Sep 10 08:27 security_hard.txt
786960 lrwxrwxrwx 1 kali kali 12 Sep 10 08:29 security_soft.txt -> security.t
xt
786937 -rw-rw-r-- 2 kali kali 60 Sep 10 08:27 security.txt
```

Figure 1: Using printf and ln to create hard and soft links.

From Figure 1, it is evident that 'security.txt' and 'security_hard.txt' share Inode 786937, proving they are the same physical file. The soft link has a unique Inode 786960.

```
(kali@kali)-[/home/kali/link_lab]
PS> ls -l
total 8
-rw-rw-r-- 2 kali kali 60 Sep 10 08:27 security_hard.txt
lrwxrwxrwx 1 kali kali 12 Sep 10 08:29 security_soft.txt -> security.txt
-rw-rw-r-- 2 kali kali 60 Sep 10 08:27 security.txt
```

Figure 2: ls -l output showing the 'l' file type for the soft link.

```
(kali@kali)-[/home/kali/link_lab]
PS> stat security.txt security_hard.txt security_soft.txt
File: security.txt
Size: 60          Blocks: 8          IO Block: 4096   regular file
Device: 8,1      Inode: 786937       Links: 2
Access: (0664/-rw-rw-r--)  Uid: ( 1000/   kali)   Gid: ( 1000/   kali)
Access: 2026-09-10 08:27:32.203942520 -0400
Modify: 2026-09-10 08:27:09.158992101 -0400
Change: 2026-09-10 08:29:08.237646179 -0400
Birth: 2026-09-10 08:27:09.118990562 -0400
File: security_hard.txt
Size: 60          Blocks: 8          IO Block: 4096   regular file
Device: 8,1      Inode: 786937       Links: 2
Access: (0664/-rw-rw-r--)  Uid: ( 1000/   kali)   Gid: ( 1000/   kali)
Access: 2026-09-10 08:27:32.203942520 -0400
Modify: 2026-09-10 08:27:09.158992101 -0400
Change: 2026-09-10 08:29:08.237646179 -0400
Birth: 2026-09-10 08:27:09.118990562 -0400
File: security_soft.txt -> security.txt
Size: 12          Blocks: 0          IO Block: 4096   symbolic link
Device: 8,1      Inode: 786960       Links: 1
Access: (0777/lrwxrwxrwx)  Uid: ( 1000/   kali)   Gid: ( 1000/   kali)
Access: 2026-09-10 08:29:38.552443054 -0400
Modify: 2026-09-10 08:29:31.292723588 -0400
Change: 2026-09-10 08:29:31.292723588 -0400
Birth: 2026-09-10 08:29:31.292723588 -0400
```

Figure 3: Stat data showing a link count of 2 for the hard link.

```
(kali@kali)-[/home/kali/link_lab]
PS> stat security.txt security_hard.txt security_soft.txt
File: security.txt
Size: 60          Blocks: 8          IO Block: 4096   regular file
Device: 8,1      Inode: 786937       Links: 2
Access: (0664/-rw-rw-r--)  Uid: ( 1000/   kali)   Gid: ( 1000/   kali)
Access: 2026-09-10 08:27:32.203942520 -0400
Modify: 2026-09-10 08:27:09.158992101 -0400
Change: 2026-09-10 08:29:08.237646179 -0400
Birth: 2026-09-10 08:27:09.118990562 -0400
File: security_hard.txt
Size: 60          Blocks: 8          IO Block: 4096   regular file
Device: 8,1      Inode: 786937       Links: 2
Access: (0664/-rw-rw-r--)  Uid: ( 1000/   kali)   Gid: ( 1000/   kali)
Access: 2026-09-10 08:27:32.203942520 -0400
Modify: 2026-09-10 08:27:09.158992101 -0400
Change: 2026-09-10 08:29:08.237646179 -0400
Birth: 2026-09-10 08:27:09.118990562 -0400
File: security_soft.txt -> security.txt
Size: 12          Blocks: 0          IO Block: 4096   symbolic link
Device: 8,1      Inode: 786960       Links: 1
Access: (0777/lrwxrwxrwx)  Uid: ( 1000/   kali)   Gid: ( 1000/   kali)
Access: 2026-09-10 08:29:38.552443054 -0400
Modify: 2026-09-10 08:29:31.292723588 -0400
Change: 2026-09-10 08:29:31.292723588 -0400
Birth: 2026-09-10 08:29:31.292723588 -0400
```

Figure 4: Stat data for the soft link showing its small size (12 bytes).

3. Part C, D, E & F: Investigation and Modification

We modified the file content and observed that changes were visible through all filenames, as they all eventually point to the same data blocks.

```
(kali㉿kali)-[/home/kali/link_lab]
PS> readlink security_hard.txt

(kali㉿kali)-[/home/kali/link_lab]
PS> readlink -f security_soft.txt
/home/kali/link_lab/security.txt
```

Figure 5: Content modification appearing across all links.

Renaming the original file demonstrated the fragility of soft links.

```
(kali㉿kali)-[/home/kali/link_lab]
PS> echo "MODified via hard link" >> security_hard.txt

(kali㉿kali)-[/home/kali/link_lab]
PS> cat security.txt security_soft.txt
Linux File System Security Lab
Confidential Security Report
Unauthorized login detected
MODified via hard link
Linux File System Security Lab
Confidential Security Report
Unauthorized login detected
MODified via hard link
```

Figure 6: Renaming target and observing the 'No such file or directory' error for the soft link.

Finally, deleting the original target showed that the data persists via the hard link, whereas the soft link remains broken.

```
(kali㉿kali)-[/home/kali/link_lab]
PS> rm incident.txt

(kali㉿kali)-[/home/kali/link_lab]
PS> cat security_hard.txt
Linux File System Security Lab
Confidential Security Report
Unauthorized login detected
MODified via hard link

(kali㉿kali)-[/home/kali/link_lab]
PS> ls -l security_soft.txt
lrwxrwxrwx 1 kali kali 12 Sep 10 08:29 security_soft.txt → security.txt

(kali㉿kali)-[/home/kali/link_lab]
PS> readlink security_soft.txt
security.txt
```

Figure 7: Data persistence through the hard link after target deletion.

4. Directory Linking Experiment

Linux restricted the creation of a hard link to the directory 'evidence'. This is a security measure to prevent recursive directory loops.

```
(kali㉿kali)-[/home/kali/link_lab]
PS> mkdir evidence

(kali㉿kali)-[/home/kali/link_lab]
PS> ln -s evidence evidence_hardlink

(kali㉿kali)-[/home/kali/link_lab]
PS> ln evidence evidence_hardlink
ln: evidence: hard link not allowed for directory

(kali㉿kali)-[/home/kali/link_lab]
PS> ln -s evidence evidence_hardlink

(kali㉿kali)-[/home/kali/link_lab]
PS> ls -l
total 8
drwxrwxr-x 2 kali kali 4096 Sep 10 08:48 evidence
lrwxrwxrwx 1 kali kali 8 Sep 10 08:47 evidence_hardlink -> evidence
-rw-rw-r-- 1 kali kali 111 Sep 10 08:41 security_hard.txt
lrwxrwxrwx 1 kali kali 12 Sep 10 08:29 security_soft.txt -> security.txt
```

Figure 8: System rejection of directory hard linking.

5. Part G: Advanced Data Sorting and Analysis

The 'sort' command is vital for security log analysis. We performed various operations including numeric sorting of failed login attempts and unique IP extraction.

```
(kali㉿kali)-[/home/kali/link_lab]
PS> bash
(kali㉿kali)-[~/link_lab]
$ cat > alerts.txt << 'EOF'
192.168.1.10 admin 5
10.0.0.8 root 12
172.16.0.5 guest 2
10.0.0.20 admin 8
EOF

sort -k3,3n alerts.txt
172.16.0.5 guest 2
192.168.1.10 admin 5
10.0.0.20 admin 8
10.0.0.8 root 12
```

Figure 9: Numeric sorting of security alerts based on the third column.

```
(kali㉿kali)-[~/link_lab]
$ cat > ports.txt << 'EOF'
22
443
8080
21
3306
EOF

(kali㉿kali)-[~/link_lab]
$ sort -n ports.txt
21
22
443
3306
8080

(kali㉿kali)-[~/link_lab]
$ sort -nr ports.txt
8080
3306
443
22
21
```

Figure 10: Numerical sorting of port numbers (-n and -nr flags).

```
(kali㉿kali)-[~/link_lab]
$ awk '{print $1}' alerts.txt | sort | uniq -c | sort -nr
1 192.168.1.10
1 172.16.0.5
1 10.0.0.8
1 10.0.0.20
```

Figure 11: Case-insensitive sorting of usernames.

```
(kali㉿kali)-[~/link_lab]
$ grep -oE '([0-9]{1,3}\.){3}[0-9]{1,3}' alerts.txt | sort | uniq
10.0.0.20
10.0.0.8
172.16.0.5
192.168.1.10

(kali㉿kali)-[~/link_lab]
$ awk '{print $1}' alerts.txt | sort | uniq -c | sort -nr | head -5
1 192.168.1.10
1 172.16.0.5
1 10.0.0.8
1 10.0.0.20
```

Figure 12: Complex pipeline (grep | sort | uniq | head) to find top suspicious IPs.

6. Conclusion

The experiment successfully demonstrated the fundamental differences between hard and symbolic links. Hard links provide data redundancy and reliability, while symbolic links offer flexibility as pointers across the filesystem. Furthermore, we mastered the 'sort' utility, which is indispensable for parsing security logs and identifying malicious patterns in system reports.

Comparison Matrix

<i>Parameter</i>	<i>Hard Link</i>	<i>Symbolic Link</i>
<i>Inode Number</i>	<i>Shared with target</i>	<i>Unique/Different</i>
<i>Cross-Filesystem</i>	<i>Not Possible</i>	<i>Supported</i>
<i>Directory Linking</i>	<i>Prohibited</i>	<i>Supported</i>
<i>File Size</i>	<i>Same as original</i>	<i>Size of path string</i>
<i>Rename Target</i>	<i>Remains functional</i>	<i>Becomes Broken</i>
<i>Delete Target</i>	<i>Data stays available</i>	<i>Becomes Dangling</i>
<i>readlink output</i>	<i>Empty</i>	<i>Path to target</i>
<i>Link Count</i>	<i>Increments</i>	<i>No effect</i>
<i>File Type</i>	<i>Regular File (-)</i>	<i>Link File (l)</i>